

Interview Packet

A tighter, PDF-ready version of the original prep guide, rewritten to be faster to scan, easier to rehearse, and easier to use the night before the interview.

Role Snapshot	What To Emphasize
Programmer IV, Information Resources Division, Austin HQ. Posted salary: \$7,745 per month. Two openings. Schedule: Monday to Friday, 8am to 5pm.	Lead the full lifecycle: analysis, design, programming, implementation, QA/QC, customer acceptance testing, and long-term support.
Technical anchors from the posting and demo: ColdFusion, Java, Hibernate, Oracle SQL, PL/SQL, JavaScript, CSS, XML, HTML.	Senior-level differentiators: JAD facilitation, requirements documentation, business cases, project plans, peer reviews, and agency IT standards.

Source Verification

Built from docs/interview_prep.md. Source last updated: 2026-03-18 13:35:05. Source hash: 517f798ac910.

Your one-sentence positioning

I translate regulated business requirements into secure, testable software, lead the work from JAD through customer acceptance testing, and communicate clearly with both technical staff and agency users.

What changed from the raw Markdown

- Condensed repetitive sections into a practical interview packet.
- Added a 30-second opener, a 90-second pitch, a 90-day plan, and cleaner closing lines.
- Reframed the tone to sound more professional and less combative than the original notes.
- Kept examples tied to the actual demo app so the prep matches the code you can discuss.

1. Interview Frame

The panel is scoring evidence, not potential. If a keyword or a delivery step is important, say it out loud. Your goal is to sound structured, senior, and easy to trust with regulated systems.

30-second opener	90-second value pitch
I am a full-stack developer who is strongest when the work sits at the intersection of legacy systems, modern application design, and stakeholder communication. The TCEQ Programmer IV role matches that well because it combines Java and Oracle development with requirements facilitation, QA/QC, peer reviews, and customer support.	In my TCEQ compliance demo, I built a system that mirrors the stack in the posting: Spring Boot, Hibernate, Oracle-style SQL, PL/SQL patterns, ColdFusion templates, JavaScript, CSS, XML, and HTML. What I would bring here is not just coding speed. I can convert requirements into design documentation, lead reviews, structure testing around acceptance criteria, and explain trade-offs clearly to program staff. That mix of technical depth and delivery discipline is what makes me a fit for a Programmer IV role.

Why TCEQ answer

I want to work on systems with a clear public mission, long service life, and real operational impact. TCEQ sits in a regulated environment where software quality, change management, and security matter, and that is the kind of work I enjoy. I also value the chance to improve critical applications over time instead of just shipping short-lived features.

Closing line

This role lines up with how I work best: start with clear requirements, design carefully, build cleanly, test against documented expectations, and support the users after go-live.

2. How To Score Well

The posting strongly suggests a standard state interview matrix. That means you want answers with an explicit sequence instead of free-form storytelling.

Step	What to do	Example phrase
1. Frame the work	Name the system, business problem, or customer need.	In a compliance tracking project, the business need was...
2. Show ownership	State your role and why you were accountable.	I led the design and coordinated the implementation plan...
3. Use process words	Say requirements documentation, design documentation, peer review, QA/QC, and customer acceptance testing when true.	We documented the requirement, reviewed the design, and tested against acceptance criteria.
4. Show standards and risk control	Mention agency IT standards, security, configuration management, or change management.	We deployed through our configuration management and change control process.

Step	What to do	Example phrase
5. Land on outcome	Finish with business value, reduced risk, or user adoption.	The result was a traceable, supportable release with clearer reporting for staff.

Use these phrases naturally	Why they matter
agency IT standards	Signals governance and security awareness.
requirements documentation and design documentation	Shows traceability from idea to implementation.
configuration management and change management	Shows disciplined releases in regulated systems.
QA/QC, peer reviews, and customer acceptance testing	Shows quality ownership beyond coding.
charter, business case, project plan, status report	Shows you can operate above an individual contributor coding lane.
JAD session and sponsor meeting	Shows stakeholder facilitation and escalation judgment.

3. Core Stories To Memorize

Memorize five stories you can bend to different questions. The goal is not to memorize every sentence. Memorize the structure, the metrics, the design choices, and the outcome.

Story 1: End-to-end delivery

Situation: You built a compliance tracking demo that had to show both technical breadth and a credible government-style delivery lifecycle.

Your move: You used a layered architecture, documented the data model and scoring rules, and tied the implementation back to requirements and design documentation.

Result: You can point to a working application, a documented architecture, and a clear explanation of how analysis became implementation.

Keywords to say: analysis, development, programming, implementation, requirements documentation, design documentation

Story 2: JAD and stakeholder alignment

Situation: Different stakeholders care about different compliance metrics, workflows, and reporting outputs.

Your move: You prepared an agenda, defined the metrics and acceptance criteria, captured decisions, and used a parking-lot approach to keep the session on track.

Result: You left with clearer scope, fewer assumptions, and artifacts the technical team could implement and the business side could review.

Keywords to say: JAD session, charter, business case, sponsor meeting, requirements documentation

Story 3: Design and architecture judgment

Situation: The posting expects technical advice, not just coding.

Your move: You explain a layered design: presentation, controller, service, repository, persistence. You justify Oracle-style DDL, sequences, constraints, indexes, and secure query patterns.

Result: You sound like someone who can design a maintainable application that fits agency standards and survives production support.

Keywords to say: technical advice, agency IT standards, layered architecture, Oracle, Hibernate, secure coding

Story 4: QA/QC and acceptance testing

Situation: The compliance scoring logic is business-critical and easy to get wrong if it is not documented and tested carefully.

Your move: You wrote tests against seeded data, covered clean and high-risk facilities, and translated the rules into a plain-language acceptance test flow.

Result: You can say the system is testable, traceable, and understandable by both reviewers and end users.

Keywords to say: QA/QC, peer review, customer acceptance testing, test plan, traceability

Story 5: Legacy modernization without breaking support

Situation: The role spans ColdFusion, Java, and long-lived agency systems.

Your move: You explain how to keep ColdFusion pages usable while moving business logic into Java services, using shared Oracle data sources and phased migration.

Result: You show respect for legacy systems while still sounding modern and pragmatic.

Keywords to say: application maintenance, customer support, ColdFusion, phased migration, change management

4. Demo App Facts You Can Reference

These details are useful because they map your answers to real files in the project instead of generic examples.

Artifact	What to say about it
docs/architecture.md	Shows the layered architecture, the data model, and the compliance scoring algorithm in a reviewer-friendly format.
ComplianceServiceTest.java	Contains 15 integration-style tests, including dashboard data, facility summaries, and high-severity compliance score scenarios.
coldfusion/permit_report.cfm	Demonstrates Oracle stored procedure calls, Query-of-Queries, PDF-style reporting, and defensive error handling in CFML.
Facility.hbm.xml plus JPA annotations	Lets you talk about legacy and modern Hibernate mapping approaches in the same codebase.
schema.sql and procedures.sql	Useful proof points for Oracle-style DDL, sequences, constraints, and PL/SQL thinking.

Stack	Likely prompt	Answer anchors
ColdFusion	How do you query safely or generate reports?	Use <cfqueryparam>, <cfstoredproc>, Query-of-Queries, HTML encoding, and a phased migration plan to Java services.
Java / Spring	How do you structure maintainable application code?	Constructor injection, service boundaries, exception handling with useful context, and tests that verify business rules.
Hibernate / JPA	What production issue do you watch for?	N+1 queries, fetch strategy, transaction boundaries, and sequence-based ID generation for Oracle.
Oracle SQL / PL/SQL	How do you tune or evolve the database safely?	Execution plans, index strategy, additive schema changes, rollback scripts, stored procedures, and validation constraints.
JavaScript / HTML / CSS / XML	How do you keep the UI secure and maintainable?	Fetch API, no raw innerHTML for user data, semantic HTML, responsive layout, and accessibility basics like focus states and contrast.
Delivery / Ops	How do you ship safely in government work?	Peer review, CI test gate, staging validation, QA/QC sign-off, change request approval, and rollback readiness.

Security-first phrasing that works well in this interview

- I validate inputs, use parameterized queries, encode output, and design for traceability.
- I assume the application may support critical infrastructure workflows, so I favor conservative release practices.
- I do not treat peer review or QA/QC as bureaucracy. In this environment they are production risk controls.

5. Written Exercise Playbook

If they include a short written or whiteboard exercise, stay calm and show your thinking. Panels often reward method over perfect syntax.

SQL bug pattern

```
SELECT f.facility_name,  
       COUNT(p.id) AS permit_count,  
       COUNT(v.id) AS violation_count  
FROM facilities f  
LEFT JOIN permits p ON f.id = p.facility_id  
LEFT JOIN violations v ON f.id = v.facility_id  
GROUP BY f.facility_name;
```

What to say

The double LEFT JOIN can multiply rows. If one facility has 3 permits and 2 violations, the joined set returns 6 rows. I would fix it with COUNT(DISTINCT ...), or by aggregating permits and violations separately before joining back to facilities.

Code review pattern

```
public List<Facility> search(String term) {  
    String sql = "SELECT * FROM facilities WHERE name LIKE '%" + term + "%'";  
    return jdbcTemplate.query(sql, new FacilityRowMapper());  
}
```

What to flag

- SQL injection risk from string concatenation.
- No null or empty handling for term.
- Leading wildcard can block normal index use.
- No pagination or result limit.
- No case-normalization strategy if the requirement expects case-insensitive search.

Design prompt pattern

Safe answer outline

- Start with charter: scope, stakeholders, timeline, constraints.
- Run a JAD session and capture requirements documentation with acceptance criteria.
- Propose the data model, service flow, notifications, reporting, and audit needs.
- Define QA/QC, customer acceptance testing, deployment, and rollback up front.
- End by naming assumptions, open questions, and the first deliverables you would produce.

6. The Questions Most Worth Asking

Prepare four to six. Prioritize the ones that reveal team maturity, modernization plans, and how much responsibility the role really carries.

Question	Why it matters
What applications would I likely support in the first 90 days?	Shows readiness to contribute early and reveals the real mix of maintenance versus new development.
Is there a formal ColdFusion-to-Java modernization roadmap?	Shows strategic thinking and tests how much architectural influence the role has.
How are JAD sessions usually run here?	Directly probes one of the strongest differentiators in the posting.
What does peer review and change management look like in practice?	Reveals delivery discipline and whether quality gates are real or informal.
How much customer support responsibility sits with the development team?	Clarifies whether you will be expected to troubleshoot directly with agency staff.
What would success look like in the first six months for this role?	Shows ownership and may surface expectations that were not obvious from the posting.

7. Day-Of Plan

Make the day easy on yourself. The goal is to reduce avoidable friction so you can spend all your energy on the interview itself.

Before	During	After
- Run the demo tests and verify you remember where the architecture and ColdFusion files live. - Practice the opener, the value pitch, and the five core stories out loud. - Bring resume copies, a notepad, a pen, and a photo ID.	- Take notes on multi-part questions and answer each part explicitly. - Keep answers near two minutes unless asked to go deeper. - Reference the demo app whenever a technical question gives you an opening.	- Thank the panel by name and ask about next steps. - Send a concise thank-you note to the HR contact within 24 hours. - Respond quickly if they ask for references or background-check materials.

Thank-you email template

Thank you for the opportunity to interview for the Programmer IV position. I appreciated learning more about the team and especially enjoyed our discussion about [specific topic]. The role's mix of technical leadership, requirements facilitation, and application delivery aligns closely with my experience, and I would be excited to contribute to TCEQ's mission.

8. Back-Pocket Answers

Scenario	A strong short answer
If you do not know a technology	I have not used that exact tool in production, but I would ramp quickly by reading the documentation, building a focused proof of concept, and validating my assumptions with someone who already supports it.
If stakeholders disagree	I would restate the competing needs, test them against the documented requirements, make the trade-off visible, and escalate to the sponsor only if the team cannot resolve it directly.
If asked why government work	I want to build software with a durable mission, clear accountability, and long-term public impact. Regulated work also fits how I like to engineer systems: carefully and transparently.
If asked about support	Support matters because agency users care about outcomes, not implementation details. I try to explain issues in workflow terms and design systems that fail gracefully and are easy to troubleshoot.

90-day plan answer

- First 30 days: learn the application landscape, release process, security constraints, and key business contacts.
- Days 31 to 60: start handling bounded enhancements, join JAD sessions, and identify a small process improvement opportunity.
- Days 61 to 90: own a more complete work item end to end and make one concrete improvement to documentation, testing, or release quality.

Final reminder

You do not need to sound flashy. You need to sound methodical, credible, and comfortable with regulated software delivery. Clear structure beats cleverness in this interview.